
Assignment 2 (Final Project – Updated on 8th May 2026)

1. Requirements

ID	Feature Category	Description
FR 1.0	Input/Parsing	The system can ingest software requirements from various sources (e.g., CSV, plain text, or direct user input).
FR 1.1	Requirement Structuring	The system can parse and tokenize raw text, identifying key components such as Input Fields, Data Ranges, Conditions, and Expected Actions.
FR 2.0	Risk Analysis & Prioritization	The system can assign a Risk Score and Test Priority (High, Medium, Low) to each imported requirement.

ID	Feature Category	Description
FR 3.0	Black-Box Test Design	The system can automatically apply and generate test cases for at least three core Black-Box techniques from ISO 29119-4 (e.g., Equivalence Partitioning, Boundary Value Analysis, Decision Tables).
FR 4.0	White-Box Test Modeling	The system can model system behavior (e.g., State Transition Diagram) and generate optimal test sequences for a chosen coverage criterion (e.g., All States).
FR 5.0	Test Oracle Generation	The system can help synthesize the Expected Result for a given requirement and specific test data.
FR 6.0	Output & Export	The system can generate test artifacts (Test Cases, Test Suites, Risk Scores) in a structured, standard format (e.g., JSON or Excel/CSV) suitable for import into Test Management Tools.
FR 7.0	Test Suite Optimization	The system includes an optimization to prioritize or minimize the generated Test Suite based on risk or coverage efficiency.

This assignment requires the development of an AI-driven AutoTestDesign tool capable of performing requirements analysis, risk assessment, and systematic test case generation, among other functions, aligning with industry standards such as ISTQB Foundation Level principles and the detailed test techniques defined in ISO/IEC/IEEE standards.

Important clarification: You will develop the AutoTestDesign tool and then use it to test a **self-chosen target application** (e.g., a web app, a calculator, a login module, a to-do list) to demonstrate the tool's effectiveness.

1.1 AutoTestDesign Requirements

The AutoTestDesign application comprises functional requirements (1.1.1) and non-functional requirements (1.1.2).

1.1.1 Functional Requirements (FR)

The system must implement at least FR 1.0, FR 1.1, FR 2.0, FR 3.0, FR 6.0, and the interactive review capability described in the “Mainly” paragraph. Other FRs (4.0, 5.0, 7.0) are optional, but implementing them will earn extra credit.

1.1.2 Non-Functional Requirements (NFR)

The non-functional requirements include but are not limited to:

- Performance
- Usability (UX/UI)
- Security
- Maintainability and Technology

1.2 Project Artifact

You must submit the following artifacts:

1. **AI-driven AutoTestDesign tool**, including source code, prompts, setup instructions (README), and a video demonstration – **(20%)**
 2. **Risk Analysis Report** for the **self-chosen target application** (not the tool itself) – (10%)
 3. **Test Plan** covering at least the following aspects – **(40%)**:
 - **Project Scope**: Background and overall objectives of the testing activity on the self-chosen target application
 - **Test Items**: Major functional and non-functional features of the target application; description of the system architecture and main components
 - **High-level Test Suite Design**: Based on the requirements of the target application and the risk analysis, select testing techniques for each test suite
 - **Schedule or Checklist**: Show test levels and objectives
 - **Organization Chart**: Explain team members' responsibilities for the testing activity using the tool
 - **Chosen Testing Framework and Rationale**: The framework used to execute tests on the target application (e.g., Selenium, JUnit, PyTest)
 - **Cost Estimation**: Cost estimation for testing the self-chosen target application using the developed AutoTestDesign tool (e.g., effort in person-hours, resource costs, comparison with manual testing)
 4. **Detailed Test Design and Execution Document** for **one major feature/module of the self-chosen target application** – (30%):
 - **Test Case Design**: Design test cases for the selected feature/module using the AutoTestDesign tool, and explain test coverage. Use multiple black-box testing techniques and also white-box techniques.
 - **Test Tool Implementation**: Choose a testing framework for executing the tests on the target application; ideally develop test scripts based on the detailed test cases above.
 - **Test Result Analysis**: Summary based on the test results.
-

Mainly: Concept → Coverage Item Identification → Coverage Strategy & Method → Test Cases and Traceability of Their Design → Prompt Design → results analysis (mapping of designed test cases to Coverage Item Identification and Coverage Strategy & Method) → improvement with evidence (newly added valid coverage items and test cases). Tools should reflect the designer's participation and interactive validation of effectiveness. The tool must allow the designer to interactively review, revise, and change design items during the above processes.

The above process must be applied to the testing of your self-chosen target application. The "designer" refers to the human tester using your AutoTestDesign tool. Your tool must support interactive modification of coverage items, strategies, and test cases.

2. Assessment Criteria

- Understanding of concepts – 10%
- Coherence of design and implementation – 20%
- Coverage and effectiveness/usefulness – 40%
- In-depth analysis (generalizability demonstration, reasoning, etc.) – 20%
- Presentation – 10%

3. Presentation

Each group has **15 minutes** to complete the project presentation, covering all aspects listed above. A Q&A session follows. Reviewers will ask questions based on the submitted documents, presentation content, and the software testing fundamentals that students are expected to have learned.

4. Submission

Each group must submit the following materials by email to the TA:

- a) **Project Artifact:** Contains all required content described in Section 1.2 above (the cover page must include the team ID, full names of all members, and student IDs).
- b) **Final Presentation PPT:** The first slide must include the team ID and full names and student IDs of all members.

The report and PPT (items 2, 3, and 4 in Section 1.2) must be submitted in PDF format, and the tool materials (item 1 in Section 1.2) must be submitted as a compressed file.

Submission and Presentation Schedule:

- **Submission Deadline:** Week 13, Friday, before 17:00
- **Presentation Dates:** Week 14–16, Tuesday/Thursday, 10:00–11:35

5. Important Clarifications for Students

To avoid confusion, please note the following:

- You are required to **develop an AutoTestDesign tool** (the “tool”).
- You must **select a separate target application** (e.g., a simple web app, a calculator, a login system, a to-do list) to be tested **using your tool**.
- The **Risk Analysis Report**, **Test Plan**, and **Detailed Test Design and Execution Document** all refer to **testing the target application**, not testing the tool itself.

- The **Cost Estimation** should estimate the cost of using your tool to test the target application, optionally compared with manual testing.
- Your tool must support **interactive design review** (allowing the tester to modify coverage items, strategies, etc.) as emphasized in the “Mainly” paragraph.
- **Performance requirements** (e.g., test case generation time must not exceed 2 seconds) are mandatory targets that you should strive to meet. If you cannot fully achieve them due to technical or resource limitations, provide a detailed analysis and improvement suggestions in your documentation.

End of Document

Scoring Table (Per Deliverable × Assessment Criteria)

Deliverable	Deliverable Weight	Concept Understanding (10%)	Coherence of Design & Implementation (20%)	Coverage & Effectiveness/Usefulness (40%)	In-depth Analysis (20%)	Presentation (10%)	Quality Score (weighted sum)	Contribution to Total Score
1. AutoTestD	20%	/100	/100	/100	/100	/100	Σ (score ×	Quality Score × 20% =

Deliverable	Deliverable Weight	Concept Understanding (10%)	Coherence of Design & Implementation (20%)	Coverage & Effectiveness/Usefulness (40%)	In-depth Analysis (20%)	Presentation (10%)	Quality Score (weighted sum)	Contribution to Total Score
Design Tool							respective weight) =	
2. Risk Analysis Report	10%	/100	/100	/100	/100	/100	=	Quality Score $\times$ 10% =
3. Test Plan	40%	/100	/100	/100	/100	/100	=	Quality Score $\times$ 40% =
4. Detailed Test Design &	30%	/100	/100	/100	/100	/100	=	Quality Score $\times$ 30% =

Deliverable	Deliverable Weight	Concept Understanding (10%)	Coherence of Design & Implementation (20%)	Coverage & Effectiveness/Usefulness (40%)	In-depth Analysis (20%)	Presentation (10%)	Quality Score (weighted sum)	Contribution to Total Score
Execution Document								
Total	100%	—	—	—	—	—	—	Sum of contributions

Example (Sample Student Scores)

Deliverable	Weight	Concept (10%)	Coherence (20%)	Coverage (40%)	In- depth (20%)	Presentation (10%)	Quality Score	Contribution
Tool	20%	85	70	80	75	90	$85 \times 0.1 +$ $70 \times 0.2 +$ $80 \times 0.4 +$ $75 \times 0.2 +$ 90×0.1 $= 79.0$	79.0×0.2 $= 15.8$
Risk Report	10%	80	75	70	80	85	$= 75.5$	75.5×0.1 $= 7.55$
Test Plan	40%	90	85	95	85	80	$= 89.0$	89.0×0.4 $= 35.6$
Detailed Test Design	30%	85	80	90	90	85	$= 86.5$	86.5×0.3 $= 25.95$
Total	100%	—	—	—	—	—	—	84.9

作业 2（期末项目 – 2026 年 5 月 8 日更新版）

1. 需求

本作业要求开发一个 AI 驱动的 AutoTestDesign 工具，能够执行需求分析、风险评估和系统化测试用例生成等功能，并符合 ISTQB 基础级原则以及 ISO/IEC/IEEE 标准中定义の詳細测试技术等行业标准。

ID	功能类别	描述
FR 1.0	输入/解析	系统可以从各种来源（例如 CSV、纯文本或直接用户输入）导入软件需求。
FR 1.1	需求结构化	系统可以解析和标记原始文本，识别关键组成部分，如输入字段、数据范围、条件和预期动作。
FR 2.0	风险分析与优先级排序	系统可以为每条导入的需求分配风险评分和测试优先级（高、中、低）。
FR 3.0	黑盒测试设计	系统可以自动应用并生成至少三种来自 ISO 29119-4 的核心黑盒技术（例如等价划分、边界值分析、决策表）的测试用例。
FR 4.0	白盒测试建模	系统可以对系统行为进行建模（例如状态转换图），并根据选定的覆盖准则（例如全状态）生成最优测试序列。

ID	功能类别	描述
FR 5.0	测试预言生成	系统可以帮助针对给定需求和特定测试数据合成预期结果。
FR 6.0	输出与导出	系统可以以结构化的标准格式（例如 JSON 或 Excel/CSV）生成测试工件（测试用例、测试套件、风险评分），以便导入测试管理工具。
FR 7.0	测试套件优化	系统包含一个优化功能，用于基于风险或覆盖效率对生成的测试套件进行优先级排序或最小化。

重要澄清： 你将开发 AutoTestDesign 工具，然后使用该工具测试一个**自选的目标应用**（例如一个 Web 应用、计算器、登录模块、待办事项列表），以验证工具的有效性。

1.1 AutoTestDesign 需求

AutoTestDesign 应用包含功能性需求（1.1.1）和非功能性需求（1.1.2）。

1.1.1 功能性需求（FR）

系统必须实现以下核心功能中的至少 FR 1.0、FR 1.1、FR 2.0、FR 3.0、FR 6.0 以及“主要内容”段落中描述的交互式审查能力。其他 FR（4.0、5.0、7.0）为可选，但实现可获得加分。

1.1.2 非功能性需求（NFR）

非功能性需求包括但不限于：

- 性能
- 可用性（UX/UI）
- 安全性
- 可维护性与技术

1.2 项目交付物

你必须提交以下交付物：

1. **AI 驱动的 AutoTestDesign 工具**，包括源代码、提示词、设置说明（README）和演示视频（**20%**）
2. 针对自选目标应用（而非工具本身）的风险分析报告（10%）
3. 测试计划，至少涵盖以下方面（**40%**）：
 - **项目范围**：针对自选目标应用的测试活动的背景和总体目标
 - **测试项**：目标应用的主要功能性和非功能性特性；系统架构及主要组件描述
 - **高级测试套件设计**：基于目标应用的需求和风险分析，为每个测试套件选择测试技术
 - **进度安排或检查清单**：展示测试级别和目标
 - **组织架构图**：解释团队成员在使用该工具进行测试活动中的职责
 - **选定的测试框架及其理由**：用于在目标应用上执行测试的框架（例如 Selenium、JUnit、PyTest）
 - **成本估算**：使用开发的 AutoTestDesign 工具测试自选目标应用的成本估算（例如人时工作量、资源成本、与手工测试的对比）
4. 针对自选目标应用的一个主要特性/模块的详细测试设计与执行文档（30%）：
 - **测试用例设计**：使用 AutoTestDesign 工具为选定的特性/模块设计测试用例，并解释测试覆盖范围。使用多种黑盒测试技术和白盒技术。
 - **测试工具实现**：选择一个测试框架在目标应用上执行测试；最好基于上述详细测试用例开发测试脚本。

- **测试结果分析：**基于测试结果的总结。

主要内容：概念 → 覆盖项识别 → 覆盖策略与方法 → 测试用例及其设计的可追溯性 → 提示设计 → 结果分析（设计的测试用例与覆盖项识别、覆盖策略与方法的映射） → 基于证据的改进（新增的有效覆盖项与测试用例）。工具应体现设计者的参与以及对有效性进行的交互式验证。该工具应允许设计者在上述过程中进行交互，以审查、修订和更改设计项。

上述过程必须应用于你自选目标应用的测试中。“设计者”指使用你的 **AutoTestDesign** 工具的人测试员。你的工具必须支持对覆盖项、策略和测试用例的交互式修改。

2. 评估标准

- 概念理解 – 10%
- 设计与实现的一致性 – 20%
- 覆盖范围与有效性/实用性 – 40%
- 深入分析（可推广性论证、推理等） – 20%
- 演示展示 – 10%

3. 演示

每组有 **15 分钟** 完成项目演示，涵盖上述所有方面。之后是问答环节。评审将根据提交的文档、演示内容以及学生应已学习过的软件测试基础知识进行提问。

4. 提交

每组必须通过电子邮件向助教提交以下材料：

- a) **项目交付物**：包含上述第 1.2 节中所有要求的内容（封面页必须包含团队 ID、所有成员的完整姓名和学号）。
- b) **最终演示 PPT**：第一张幻灯片必须包含团队 ID 以及所有成员的完整姓名和学号。

报告和 **PPT**（第 1.2 节中的第 2、3、4 项）必须以 **PDF** 格式提交，工具材料（第 1.2 节中的第 1 项）必须以压缩文件形式提交。

提交与演示时间安排：

- **提交截止日期**：第 13 周，周五，下午 17:00 之前
- **演示日期**：第 14–16 周，周二/周四，10:00–11:35

5. 对学生的重要澄清

为避免混淆，请注意以下事项：

- 你需要开发一个 **AutoTestDesign** 工具（“工具”）。
- 你必须**选择**一个独立的目标应用（例如一个简单的 Web 应用、计算器、登录系统、待办事项列表）**使用你的工具**对其进行测试。
- **风险分析报告、测试计划和详细测试设计与执行文档**均指**测试目标应用**，而不是测试工具本身。

- **成本估算**应估算使用你的工具测试目标应用的成本，可选择与手工测试进行对比。
 - 你的工具必须支持**交互式设计审查**（允许测试人员修改覆盖项、策略等），如“主要内容”段落所强调。
 - **性能要求**（例如测试用例生成时间不超过 2 秒）为必须努力满足的指标。若因技术或资源限制无法完全达到，请在文档中提供详细分析和改进建议。
-

文档结束

评分表格（按交付物 × 评估标准拆分）

交付物	交付物权重	概念理解 (10%)	设计一致性 (20%)	覆盖范围 (40%)	深入分析 (20%)	演示展示 (10%)	质量得分 (加权)	对总分贡献分
1. AutoTestDesign 工具	20%	/100	/100	/100	/100	/100	Σ （左边各项得分×对应权重） =	质量得分 × 20% =
2. 风险分析报告	10%	/100	/100	/100	/100	/100	同上计算 =	质量得分 × 10% =
3. 测试计划	40%	/100	/100	/100	/100	/100	同上计算 =	质量得分 × 40% =
4. 详细测试设计与执行文档	30%	/100	/100	/100	/100	/100	同上计算 =	质量得分 × 30% =
总分	100%	—	—	—	—	—	—	Σ 各贡献分

填写示例（假设某学生成绩）

交付物	权重	概念 (10%)	设计 (20%)	覆盖 (40%)	深入 (20%)	演示 (10%)	质量得分	贡献分
工具	20%	85	70	80	75	90	$85 \times 0.1 + 70 \times 0.2 + 80 \times 0.4 + 75 \times 0.2 + 90 \times 0.1 = 79.0$	$79.0 \times 0.2 = 15.8$
风险报告	10%	80	75	70	80	85	$= 75.5$	$75.5 \times 0.1 = 7.55$
测试计划	40%	90	85	95	85	80	$= 89.0$	$89.0 \times 0.4 = 35.6$
详细测试设计	30%	85	80	90	90	85	$= 86.5$	$86.5 \times 0.3 = 25.95$
总分	100%	—	—	—	—	—	—	84.9
